

C# ve .NET Uygulama Geliştirme

C# dilini ve modern .NET ekosistemini temel sözdiziminden web API, veri erişimi, asenkron programlama ve test yaklaşımına kadar uygulamalı biçimde öğreten sentetik ders dokümanı.

Seviye

Başlangıç -> Orta

Önerilen süre

18-24 saat

Belge yapısı

15 sayfa / ders modülleri + uygulama

Kapsam: C# | Modern .NET | LINQ | ASP.NET Core

Amaç: Tek başına çalışılabilir, örnek ve mini uygulamalar içeren ders dokümanı.

MODÜL 0

Ders Haritası ve Öğrenme Çıktıları

Bu ders, C# sözdizimini yalnızca ezberletmek yerine .NET üzerinde gerçek uygulama geliştirme akışını öğretmeyi hedefler. Her modül sonunda küçük bir uygulama veya kontrol sorusu bulunur.

15 sayfalık yapı

Sayfa	Tema	Çıktı
3	CLR, proje ve tip sistemi	Bir .NET projesinin nasıl çalıştığını açıklar
4-6	Dil + OOP	Okunabilir sınıf ve metod tasarlar
7-10	Collections, LINQ, async	Veriyi işler ve paralel I/O yönetir
11-14	API, veri, test	Katmanlı uygulama geliştirir
15	Bitirme görevi	Küçük bir servis tasarlar

Ön koşul

Temel programlama mantığı yeterlidir. C/C++ veya Java bilenler ilk bölümleri daha hızlı geçebilir.

Öğrenme çıktıları

- Değer ve referans tipleri arasındaki farkı uygular.
- Sınıf, record, interface ve kalıtımı doğru yerde kullanır.
- LINQ ile koleksiyonları filtreler, dönüştürür ve gruplar.
- async/await ile bloklamayan I/O akışları kurar.
- ASP.NET Core ile REST API geliştirir ve EF Core ile veri kalıcılığı sağlar.
- xUnit benzeri test yaklaşımıyla kritik davranışları doğrular.

MODÜL 1

.NET Çalışma Modeli: SDK, CLR ve Proje Yapısı

.NET SDK derleme, paket yönetimi ve çalıştırma araçlarını sağlar. C# kaynak kodu önce ara dile (IL) derlenir; CLR bu kodu JIT derleyerek hedef işlemcide çalıştırır. Garbage Collector, yönetilen belleğin büyük bölümünü otomatik yönetir.

Bileşen	Rol
dotnet CLI	Proje oluşturma, derleme, test ve çalıştırma
CLR	IL yürütme, JIT, GC, exception altyapısı
NuGet	Paket ve bağımlılık yönetimi
.csproj	Target framework, paketler ve derleme ayarları

KOMUT SATIRI ÖRNEĞİ

```
dotnet new console -n DemoApp
cd DemoApp
dotnet run
dotnet add package Serilog
dotnet test
```

Uygulama

Yeni bir console projesi oluşturun. Program.cs içine işletim sistemi, tarih ve kullanıcıdan alınan adı yazdıran üç satırlık çıktı ekleyin.

Kritik kavram

- Managed code: CLR denetiminde çalışan kod.
- Target Framework Moniker: örneğin net8.0; projenin hedeflediği .NET çalışma zamanı sözleşmesini belirtir.
- Release derlemesi, Debug derlemesine göre daha fazla optimizasyon içerir.

MODÜL 2

Tipler, Değişkenler ve Kontrol Akışı

C# güçlü ve statik tipli bir dildir. var anahtar sözcüğü tip güvenliğini kaldırmaz; derleyici tipi sağ taraftan çıkarır. Nullable reference types açıldığında olası null kullanımları derleme aşamasında görünür hale gelir.

TEMEL SÖZDİZİMİ

```
int count = 12;
decimal price = 149.90m;
string? note = null;
var total = count * price;

if (total > 1000m)
    Console.WriteLine("High value");
else
    Console.WriteLine("Standard");
```

Yapı	Ne zaman?
if / switch	Koşullu iş akışı
for / foreach	Sayaç veya koleksiyon dolaşımı
while	Koşul devam ettiği sürece yineleme
switch expression	Değer döndüren kısa karar tabloları

Mini görev

1-100 arasındaki sayılarda 3 ile bölünenlere Fizz, 5 ile bölünenlere Buzz, her ikisine bölünenlere FizzBuzz yazdırın.

MODÜL 3

Metotlar, Parametreler ve Pattern Matching

İyi metotlar küçük, tek amaçlı ve adı davranışı açıklayan yapılardır. C#; optional parameters, named arguments, out parametreleri, tuple dönüşleri ve pattern matching gibi okunabilirliği artıran birçok araç sunar.

METOT + TUPLE + SWITCH EXPRESSION

```
static (decimal net, decimal tax) Calculate(decimal gross, decimal rate = 0.20m)
{
    var tax = gross * rate;
    return (gross - tax, tax);
}

var (net, tax) = Calculate(1200m);
string label = net switch
{
    > 1000m => "A",
    > 500m  => "B",
    _      => "C"
};
```

- ref: mevcut değişkene referans üzerinden erişir.
- out: metodun kesin olarak değer atayacağı çıkış parametresidir.
- params: değişken sayıda argüman almayı kolaylaştırır.
- is ve property patterns: çalışma zamanı tip/değer kontrolünü okunabilir hale getirir.

Tasarım ilkesi

Bir metot 20-30 satırı düzenli biçimde aşıyor ve birden fazla iş yapıyorsa daha küçük metotlara bölmeyi değerlendirin.

MODÜL 4

Nesne Yönelimli Tasarım: class, record, interface

C# nesne yönelimli programlamayı sınıflar, erişim belirleyicileri, interface, abstract class ve records üzerinden destekler. Amaç her şeyi sınıflandırmak değil; davranış ve veriyi tutarlı sorumluluklar altında toplamaktır.

INTERFACE VE RECORD

```
public interface INotifier
{
    Task SendAsync(string message);
}

public record Order(int Id, decimal Total);

public sealed class ConsoleNotifier : INotifier
{
    public Task SendAsync(string message)
    {
        Console.WriteLine(message);
        return Task.CompletedTask;
    }
}
```

Araç	Tercih nedeni
class	Kimlik ve davranış taşıyan nesne
record	Değer odaklı, immutable veri modeli
interface	Sözleşme ve gevşek bağlılık
abstract class	Ortak temel davranış + kısmi sözleşme

SOLID bağlantısı

Bağımlılığı somut sınıfa değil interface sözleşmesine vermek, test edilebilirliği ve değiştirilebilirliği artırır.

MODÜL 5

Generics ve Koleksiyonlar

Generics, farklı tiplerde aynı algoritmayı tip güvenli biçimde yeniden kullanmayı sağlar. List<T>, Dictionary<TKey,TValue>, HashSet<T> ve Queue<T> günlük uygulamalarda en sık kullanılan koleksiyonlardandır.

GENERIC METOT VE KOLEKSİYONLAR

```
static T? FirstOrDefaultSafe<T>(IEnumerable<T> items, Func<T, bool> predicate)
{
    foreach (var item in items)
        if (predicate(item)) return item;
    return default;
}

var prices = new List<decimal> { 10, 25, 40 };
var map = new Dictionary<int, string> { [1] = "Ada" };
```

Koleksiyon	Yaklaşık kullanım
List<T>	Sıralı dinamik liste
Dictionary<K,V>	Anahtar ile hızlı erişim
HashSet<T>	Tekil elemanlar ve üyelik testi
Queue<T> / Stack<T>	FIFO / LIFO iş akışı

Mini görev

Bir ürün listesinde tekrar eden kategori adlarını HashSet ile tekilleştirin; ürünleri Id anahtarıyla Dictionary içine yerleştirin.

MODÜL 6

LINQ ve Lambda İfadeleri

LINQ, koleksiyon sorgularını veri kaynağından bağımsız ve zincirlenebilir işlemler halinde ifade eder. Where, Select, OrderBy, GroupBy, Any, All ve Aggregate temel yapı taşlarıdır. IEnumerable üzerinde pek çok işlem deferred execution ile ihtiyaç anında çalışır.

LINQ ZİNCİRİ

```
var result = orders
    .Where(o => o.Total >= 1000m)
    .GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Total = g.Sum(x => x.Total) })
    .OrderByDescending(x => x.Total)
    .Take(5)
    .ToList();
```

- Where filtreler; Select şekil dönüştürür.
- SelectMany iç içe koleksiyonları düzleştirir.
- ToList / ToArray sorguyu o noktada gerçekleştirir.
- EF Core ile LINQ ifadesi SQL sorgusuna çevrilebilir; her .NET metodu SQL karşılığına sahip olmayabilir.

Performans notu

Bir koleksiyonu tekrar tekrar enumerate etmek pahalıysa sonucu bir kez materyalize etmeyi düşünün. Ancak gereksiz ToList çağrıları da bellek maliyeti yaratır.

MODÜL 7

Hata Yönetimi, Nullable ve Logging

Exception, olağan dışı durumları çağrı zincirinde yukarı taşır. try/catch her satırı çevrelemek için değil, hatayı anlamlı biçimde ele alabileceğiniz sınırdaki kullanılmalıdır. Beklenen iş kuralları için exception yerine sonuç tipi veya doğrulama tercih edilebilir.

STRUCTURED LOGGING

```
try
{
    await service.ProcessAsync(id);
}
catch (OperationCanceledException)
{
    logger.LogInformation("Operation cancelled: {Id}", id);
}
catch (Exception ex)
{
    logger.LogError(ex, "Processing failed: {Id}", id);
    throw;
}
```

Pratik	Neden?
throw;	Orijinal stack trace korunur
using / await using	Dispose gerektiren kaynaklar kapanır
string?	Null olasılığı açık hale gelir
ILogger<T>	Aranabilir ve yapılandırılmış log üretir

Kaçınılacak hata

catch (Exception) { } gibi hatayı tamamen yutan bloklar teşhis kabiliyetini yok eder.

MODÜL 8

Asenkron Programlama: Task ve async/await

async/await özellikle dosya, ağ ve veritabanı gibi I/O bekleyen işlerde thread bloklamadan ölçeklenebilirlik sağlar. CPU yoğun iş için ise paralellik ayrı değerlendirilmelidir. async void yalnızca event handler gibi özel yerlerde kullanılmalıdır.

ASENKRON I/O

```
public async Task<string> DownloadAsync(HttpClient client, string url, CancellationToken ct)
{
    using var response = await client.GetAsync(url, ct);
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync(ct);
}

var texts = await Task.WhenAll(urls.Select(u => DownloadAsync(client, u, ct)));
```

- CancellationToken iptal sinyalini zincir boyunca taşır.
- Task.WhenAll bağımsız işleri beraber bekler.
- ConfigureAwait çoğunlukla kütüphane kodunda önemlidir; ASP.NET Core tarafında klasik UI synchronization context bulunmaz.
- SemaphoreSlim ile eşzamanlı iş sayısı sınırlandırılabilir.

Mini görev

10 URL işleyeceğinizi varsayın. Aynı anda en fazla 3 istek çalışacak şekilde SemaphoreSlim tabanlı akış tasarlayın.

MODÜL 9

Dosya, JSON ve HTTP İstemcisi

Gerçek uygulamalar dosya sistemi, JSON ve HTTP ile sıkça etkileşir. System.Text.Json modern .NET içinde yerleşik JSON serileştirme sağlar. HttpClient tekrar kullanılmalı; her istekte yeni instance oluşturmak socket tüketimine yol açabilir.

I/O ÖRNEĞİ

```
public record AppConfig(string ApiUrl, int TimeoutSeconds);

var json = await File.ReadAllTextAsync("app.json");
var cfg = JsonSerializer.Deserialize<AppConfig>(json)!;

using var client = new HttpClient { BaseAddress = new Uri(cfg.ApiUrl) };
var payload = await client.GetStringAsync("/health");
```

API	Amaç
File / FileStream	Dosya okuma-yazma
Path	Platform bağımsız yol işlemleri
JsonSerializer	JSON serialize/deserialize
HttpClient	HTTP istekleri

Güvenlik

API anahtarlarını kaynak koda yazmayın. Development ortamında user-secrets; üretimde güvenli secret store veya çevre değişkenleri kullanın.

MODÜL 10

EF Core ile Veri Erişimi

Entity Framework Core bir ORM katmanıdır. DbContext değişiklik takibi ve sorgu üretimini yönetir. Migration yaklaşımı, modeldeki değişiklikleri veritabanı şemasına kontrollü biçimde taşır.

EF CORE SORGUSU

```
public class AppDb : DbContext
{
    public DbSet<Product> Products => Set<Product>();
}

var expensive = await db.Products
    .AsNoTracking()
    .Where(p => p.Price > 1000m)
    .OrderBy(p => p.Name)
    .ToListAsync(ct);
```

- AsNoTracking salt-okuma senaryolarında tracking maliyetini azaltır.
- Include ilişkili veriyi eager loading ile getirir; gereksiz Include büyük sorgular oluşturabilir.
- Migration dosyaları kaynak kontrolünde tutulmalıdır.
- N+1 sorgu problemini SQL logları ve profiling ile takip edin.

Mini görev

Product(Id, Name, Price, Category) modeli için Price alanına göre filtreleyen bir repository metodu yazın; DTO döndürmeyi tercih edin.

MODÜL 11

ASP.NET Core ile Minimal REST API

ASP.NET Core, dependency injection, configuration, logging ve middleware özelliklerini tek çatı altında sunar. Minimal API, küçük servisler ve prototipler için düşük törenselle bir başlangıç sağlar.

MINIMAL API

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDb>();
var app = builder.Build();

app.MapGet("/products/{id:int}", async (int id, AppDb db) =>
    await db.Products.FindAsync(id) is { } p
    ? Results.Ok(p)
    : Results.NotFound());

app.Run();
```

HTTP	Tipik kullanım
GET	Kaynak oku
POST	Yeni kaynak oluştur
PUT/PATCH	Kaynağı güncelle
DELETE	Kaynağı sil

API tasarımı

Doğru status code, tutarlı hata gövdesi, doğrulama ve idempotency kuralları API kalitesini doğrudan etkiler.

MODÜL 12

Test, Debugging ve Katmanlama

Testin amacı satır sayısını artırmak değil, kritik davranışların değişikliklerden sonra da korunmasını sağlamaktır. Unit test hızlı ve izole olmalı; integration test dış bileşenlerle gerçek entegrasyonu doğrulamalıdır.

XUNIT TARZI TEST

```
[Fact]
public void Discount_is_10_percent_for_premium_customer()
{
    var sut = new PriceService();
    var result = sut.ApplyDiscount(100m, CustomerType.Premium);
    Assert.Equal(90m, result);
}
```

- Arrange - Act - Assert yapısı testi okunabilir tutar.
- Breakpoint, conditional breakpoint ve watch ifadeleri debugger kullanımını güçlendirir.
- Domain, application ve infrastructure katmanlarını ayırmak bağımlılık yönünü kontrol eder.
- Mock yalnızca gerçek bağımlılığı kullanmanın testi yavaş veya nondeterministic yaptığı yerlerde değerlidir.

Kod inceleme kontrolü

Metot adları niyeti açıklıyor mu? Exception politikası tutarlı mı? CancellationToken iletiliyor mu? Veri sorguları gereksiz yük getiriyor mu?

MODÜL 13 - FINAL

Bitirme Projesi: Görev Takip API

Amaç, ders boyunca kullanılan kavramları tek bir küçük serviste birleştirmektir. Proje: kullanıcıların görev oluşturduğu, durum güncellediği ve filtrelediği bir Task Tracking API.

Zorunlu özellikler

- TaskItem: Id, Title, Description, Priority, Status, DueDate alanları.
- GET /tasks?status=&priority= filtreleme ve sıralama.
- POST /tasks doğrulama; boş başlık kabul edilmemeli.
- EF Core ile SQLite kalıcılığı ve migration.
- ILogger ile hata ve önemli olay loglama.
- En az 5 unit test ve 2 integration test.
- Opsiyonel: JWT kimlik doğrulama, pagination ve OpenAPI açıklamaları.

Kısa değerlendirme

Soru	Beklenen odak
1. record ne zaman class yerine uygundur?	Değer semantiği / immutable DTO
2. Task.WhenAll ne sağlar?	Bağımsız asenkron işleri beraber bekleme
3. AsNoTracking neden kullanılır?	Salt-okuma sorgu maliyeti
4. interface neyi iyileştirir?	Soyutlama, test edilebilirlik
5. throw; ile throw ex; farkı?	Stack trace korunumu

Başarı ölçütü

Proje derlenmeli, testler geçmeli, API hata durumlarında anlamlı HTTP sonucu döndürmeli ve kod tekrarını sınırlamalıdır.